

GESTION DES NOTES

Rapport de Projet

Réalisé par :

EL MAZYANI MOHAMED ILYAS
HAMZA ABAGAR
BRIK AOUSSAR

Encadré par :

Dr. Khammal

Version : 1.0

Date : 18 juin 2026

Table des matières

1	Introduction	6
1.1	Présentation du projet	6
1.2	Objectifs du projet	6
1.3	Acteurs du système	6
1.4	Technologies utilisées	6
2	Architecture du projet	7
2.1	Architecture générale	7
2.2	Organisation des packages	7
2.3	Flux de données	7
3	Diagramme de classe	8
3.1	Description des classes	8
3.2	Structure des classes	8
3.2.1	Classe Utilisateur	8
3.2.2	Classe Etudiant	9
3.2.3	Classe Enseignant	10
3.2.4	Classe Module	11
3.2.5	Classe SousModule	12
3.2.6	Classe Note	13
3.2.7	Classe Promotion	14
4	Structure de la base de données	16
4.1	Schéma de la base de données	16
4.2	Relations entre les tables	16
4.3	Requêtes principales	16
4.3.1	Authentification	16
4.3.2	Récupération des étudiants par promotion	17
4.3.3	Récupération des notes d'un étudiant	17
4.3.4	Insertion/Mise à jour d'une note	17
5	Règles de validation	18
5.1	Règlement d'évaluation	18
5.2	Calcul des moyennes	18
5.3	Algorithmes clés	20
5.3.1	Validation d'un module	20
5.3.2	Calcul de la moyenne d'un module	20
5.3.3	Validation de l'année	20
6	Interface utilisateur	22
6.1	Page de connexion	22
6.2	Interface principale	23
6.3	Gestion des étudiants	24
6.4	Gestion des enseignants	25
6.5	Gestion des promotions	25
6.6	Gestion des modules	25

6.7	Saisie des notes (Enseignant)	25
6.8	Consultation des notes (Étudiant)	26
6.9	Validation de l'année (Responsable)	26
6.10	Statistiques (Planning et Responsable)	26
6.11	Progression (Étudiant)	27
6.12	Structure du menu par rôle	27
7	Fonctionnalités détaillées	28
7.1	Gestion des étudiants	28
7.1.1	Ajout d'un étudiant	28
7.1.2	Modification d'un étudiant	28
7.1.3	Archivage	28
7.1.4	Suppression définitive	28
7.2	Gestion des enseignants	28
7.2.1	Ajout d'un enseignant	28
7.2.2	Modification	29
7.2.3	Suppression	29
7.3	Saisie des notes	29
7.3.1	Édition en ligne	29
7.3.2	Sauvegarde	29
7.3.3	Import Excel	29
7.3.4	Export Excel	29
7.4	Validation de l'année	30
7.4.1	Calcul automatique	30
7.4.2	Décision du jury	30
7.5	Statistiques	30
7.5.1	Classement	30
7.5.2	Graphiques	30
7.5.3	Bilan	30
8	Données initiales de test	31
8.1	Utilisateurs	31
8.2	Étudiants	31
8.3	Promotions	31
8.4	Modules et Sous-modules	31
8.5	Notes de test	32
9	Description détaillée des captures d'écran	33
9.1	Capture 1 : Classe Utilisateur	33
9.2	Capture 2 : Classe Enseignant	34
9.3	Capture 3 : Classe Etudiant	35
9.4	Capture 4 : Classe Module	36
9.5	Capture 5 : Classe Note	37
9.6	Capture 6 : Classe Promotion	38
9.7	Capture 7 : Classe SousModule	39
9.8	Captures 8, 9, 10 : ReglementEvaluation	40
9.9	Capture 11 : Page de connexion	42
9.10	Capture 12 : Interface principale	43
9.11	Capture 13 : Statistiques	44

10 Conclusion	45
10.1 Bilan du projet	45
10.2 Points forts	45
10.3 Fonctionnalités réalisées	45
10.4 Améliorations possibles	45
10.5 Équipe de développement	46
10.6 Remerciements	46
11 Annexes	47
11.1 Configuration du projet	47
11.2 Dépendances principales	47
11.3 Structure des fichiers	47
11.4 Commandes utiles	48
11.4.1 Compilation avec JavaFX	48
11.4.2 Exécution	48
11.4.3 Base de données MySQL	48

Table des figures

1	Architecture en couches du système	7
2	Classe Utilisateur	9
3	Classe Etudiant	10
4	Classe Enseignant	11
5	Classe Module	12
6	Classe SousModule	13
7	Classe Note	14
8	Classe Promotion	15
9	Relations entre les tables	16
10	Classe ReglementEvaluation - Calcul des moyennes	18
11	Suite du calcul des moyennes	19
12	Validation et classe de résultat	19
13	Interface de connexion	22
14	Interface principale - Gestion des étudiants	23
15	Gestion des étudiants	24
16	Fonctionnalités de validation	26
17	Classe Utilisateur - Gestion des comptes	33
18	Classe Enseignant	34
19	Classe Etudiant	35
20	Classe Module	36
21	Classe Note	37
22	Classe Promotion	38
23	Classe SousModule	39
24	ReglementEvaluation - Partie 1	40
25	ReglementEvaluation - Partie 2	41
26	ReglementEvaluation - Partie 3	41
27	Page de connexion	42
28	Interface principale - Gestion des étudiants	43
29	Statistiques et rapports	44

Liste des tableaux

1	Classes principales du projet	8
2	Tables de la base de données	16
3	Fonctionnalités par rôle	27
4	Utilisateurs de test	31
5	Étudiants de test (extrait)	31
6	Promotions de test	31
7	Modules de test	31
8	Sous-modules de test (extrait)	32
9	Notes de test (extrait - Étudiant 1)	32

1 Introduction

1.1 Présentation du projet

Ce projet consiste en une application de gestion des notes destinée aux établissements d'enseignement supérieur. Développée par **EL MAZYANI MOHAMED ILYAS, HAMZA ABAGAR** et **BRIK AOUSSAR**, l'application permet aux différents acteurs (étudiants, enseignants, responsables de filière et personnel du planning) de gérer efficacement les notes, les promotions, les modules et les évaluations.

1.2 Objectifs du projet

- **Centraliser** la gestion des notes des étudiants dans une base de données unique
- **Automatiser** le calcul des moyennes et des validations selon le règlement d'évaluation
- **Offrir** une interface intuitive et moderne pour chaque type d'utilisateur
- **Assurer** le suivi de la progression académique des étudiants
- **Faciliter** l'import/export des données via Excel
- **Garantir** la sécurité des données avec un système d'authentification

1.3 Acteurs du système

- **PLANNING** : Gestion complète des étudiants, enseignants, promotions, modules et notes
- **RESPONSABLE_FILIERE** : Gestion des étudiants, validation des années et consultation des statistiques
- **ENSEIGNANT** : Saisie et gestion des notes des sous-modules qui lui sont assignés
- **ETUDIANT** : Consultation des notes et suivi de la progression

1.4 Technologies utilisées

- **Java** : Langage de programmation principal (JDK 17+)
- **JavaFX** : Framework pour l'interface utilisateur (version 26.0.1)
- **MySQL** : Base de données relationnelle (version 8.0+)
- **Apache POI** : Import/Export des fichiers Excel (version 5.2.5)
- **MySQL Connector/J** : Pilote JDBC pour MySQL (version 9.7.0)

2 Architecture du projet

2.1 Architecture générale

Le projet suit une architecture en couches avec une séparation claire entre l'interface utilisateur, la couche métier et la couche d'accès aux données.

Interface Utilisateur (JavaFX)
Couche Métier (Java)
Couche Données (MySQL)

FIGURE 1 – Architecture en couches du système

2.2 Organisation des packages

- **modele/** : Contient toutes les classes métier (Etudiant, Enseignant, Note, etc.)
- **InterfaceFx.java** : Classe principale de l'interface utilisateur JavaFX
- **StockageMySQL.java** : Classe d'accès à la base de données
- **ReglementEvaluation.java** : Implémentation des règles de validation
- **Principal.java** : Point d'entrée de l'application
- **lib/** : Bibliothèques externes (MySQL Connector, Apache POI, etc.)

2.3 Flux de données

1. L'utilisateur interagit avec l'interface JavaFX
2. Les actions sont traitées par les méthodes de la classe InterfaceFx
3. La couche métier (classes du package modele) traite les données
4. La classe StockageMySQL effectue les opérations en base de données
5. Les résultats sont renvoyés à l'interface utilisateur

3 Diagramme de classe

3.1 Description des classes

TABLE 1 – Classes principales du projet

Classe	Rôle
Utilisateur	Gestion des comptes utilisateurs et authentification
Etudiant	Représentation d'un étudiant avec ses informations personnelles
Enseignant	Représentation d'un enseignant avec ses assignations
Promotion	Gestion des promotions et filières
Module	Gestion des modules d'enseignement
SousModule	Gestion des sous-modules avec coefficients
Note	Gestion des notes (initiale et rattrapage)
StockageMySQL	Accès à la base de données avec toutes les méthodes CRUD
InterfaceFx	Interface utilisateur JavaFX complète
ReglementEvaluation	Implémentation des règles de validation

3.2 Structure des classes

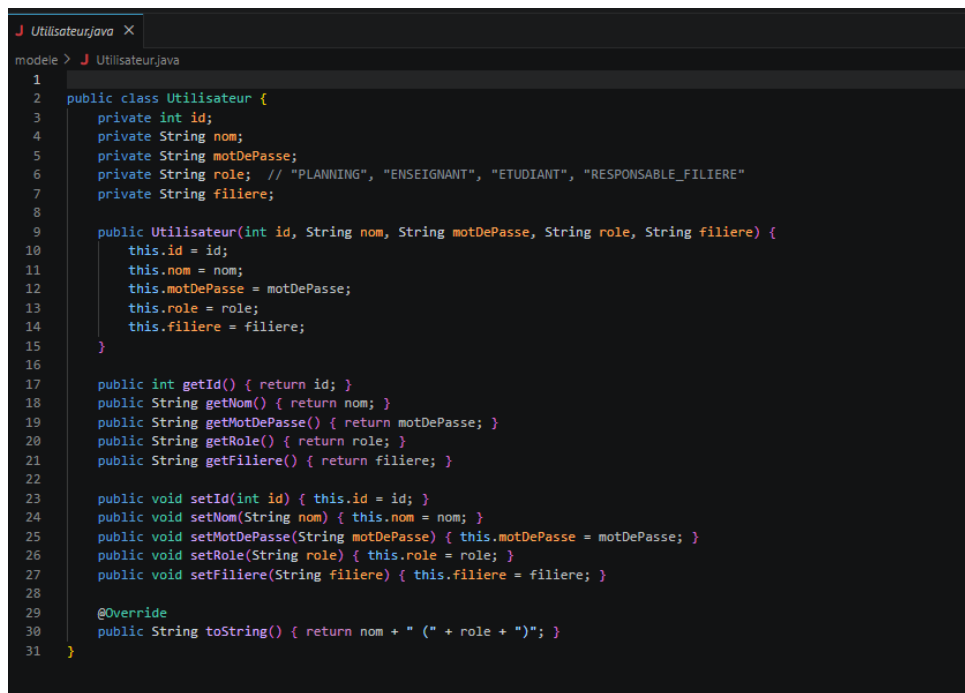
3.2.1 Classe Utilisateur

```

1 public class Utilisateur {
2     private int id;                // Identifiant unique
3     private String nom;            // Nom d'utilisateur (login)
4     private String motDePasse;    // Mot de passe (stock en clair)
5     private String role;          // PLANNING, ENSEIGNANT, ETUDIANT,
        RESPONSABLE_FILIERE
6     private String filiere;        // Fili re pour les responsables
        (NULL sinon)
7     // Getters et Setters
8 }

```

Listing 1 – Classe Utilisateur - Gestion des comptes



```

1 public class Utilisateur {
2     private int id;
3     private String nom;
4     private String motDePasse;
5     private String role; // "PLANNING", "ENSEIGNANT", "ETUDIANT", "RESPONSABLE_FILIERE"
6     private String filiere;
7
8     public Utilisateur(int id, String nom, String motDePasse, String role, String filiere) {
9         this.id = id;
10        this.nom = nom;
11        this.motDePasse = motDePasse;
12        this.role = role;
13        this.filiere = filiere;
14    }
15
16    public int getId() { return id; }
17    public String getNom() { return nom; }
18    public String getMotDePasse() { return motDePasse; }
19    public String getRole() { return role; }
20    public String getFiliere() { return filiere; }
21
22    public void setId(int id) { this.id = id; }
23    public void setNom(String nom) { this.nom = nom; }
24    public void setMotDePasse(String motDePasse) { this.motDePasse = motDePasse; }
25    public void setRole(String role) { this.role = role; }
26    public void setFiliere(String filiere) { this.filiere = filiere; }
27
28    @Override
29    public String toString() { return nom + " (" + role + ")"; }
30 }

```

FIGURE 2 – Classe Utilisateur

Fonctionnalités :

- Authentification des utilisateurs via login/mot de passe
- Gestion des rôles et permissions
- Association des responsables à une filière spécifique

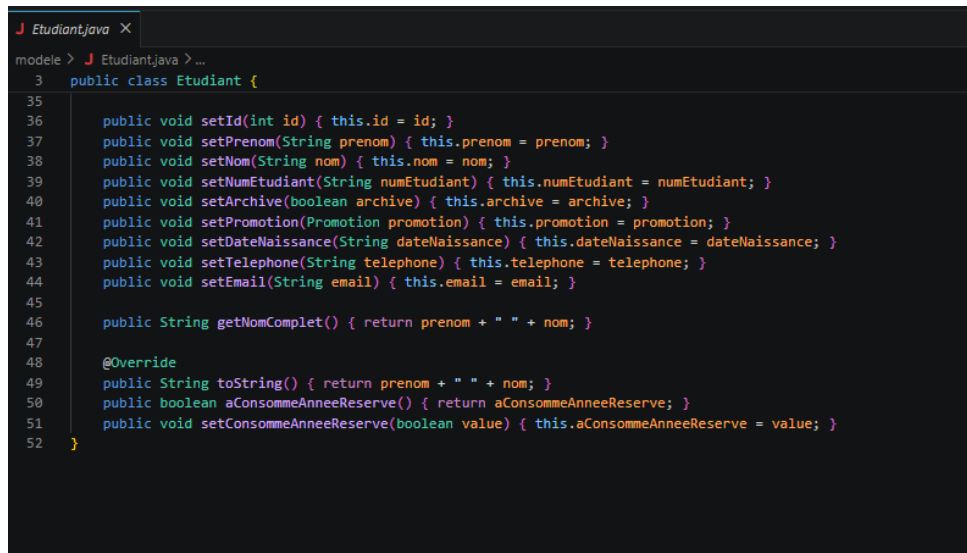
3.2.2 Classe Etudiant

```

1 public class Etudiant {
2     private int id; // Identifiant unique
3     private String prenom; // Pr nom de l' tudiant
4     private String nom; // Nom de l' tudiant
5     private String numEtudiant; // Num ro tudiant (format:
6     A123)
7     private boolean archive; // Statut archiv ou actif
8     private Promotion promotion; // Promotion associ e
9     private String dateNaissance; // Date de naissance (DD/MM/YYYY)
10    private String telephone; // Num ro de t l phone
11    private String email; // Adresse email
12    // Getters et Setters
13 }

```

Listing 2 – Classe Etudiant - Représentation d'un étudiant

A screenshot of a Java IDE window titled 'Etudiant.java'. The code defines a public class 'Etudiant' with various methods. Line 35 starts the class definition. Lines 36-45 are setter methods for 'id', 'prenom', 'nom', 'numEtudiant', 'archive', 'promotion', 'dateNaissance', 'telephone', and 'email'. Line 46 is a getter for 'nomComple' which concatenates 'prenom' and 'nom'. Line 48 is an '@Override' annotation. Line 49 is a 'toString' method. Line 50 is a 'aConsommeAnneeReserve' method. Line 51 is a 'setConsommeAnneeReserve' method. Line 52 closes the class.

```
3 public class Etudiant {
35
36     public void setId(int id) { this.id = id; }
37     public void setPrenom(String prenom) { this.prenom = prenom; }
38     public void setNom(String nom) { this.nom = nom; }
39     public void setNumEtudiant(String numEtudiant) { this.numEtudiant = numEtudiant; }
40     public void setArchive(boolean archive) { this.archive = archive; }
41     public void setPromotion(Promotion promotion) { this.promotion = promotion; }
42     public void setDateNaissance(String dateNaissance) { this.dateNaissance = dateNaissance; }
43     public void setTelephone(String telephone) { this.telephone = telephone; }
44     public void setEmail(String email) { this.email = email; }
45
46     public String getNomComple() { return prenom + " " + nom; }
47
48     @Override
49     public String toString() { return prenom + " " + nom; }
50     public boolean aConsommeAnneeReserve() { return aConsommeAnneeReserve; }
51     public void setConsommeAnneeReserve(boolean value) { this.aConsommeAnneeReserve = value; }
52 }
```

FIGURE 3 – Classe Etudiant

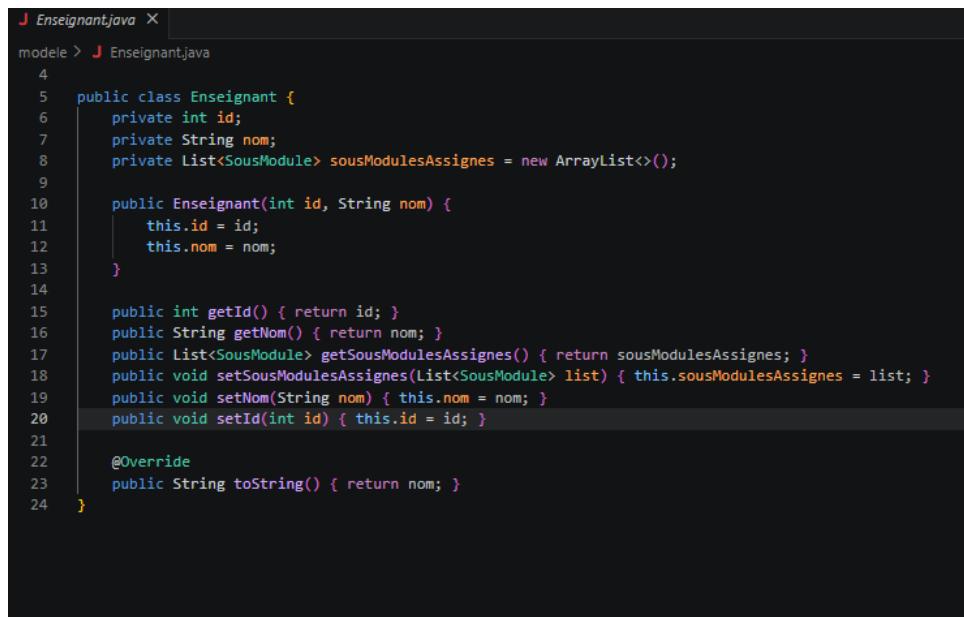
Fonctionnalités :

- Gestion complète des informations personnelles
- Association à une promotion
- Archivage/Restauration des étudiants
- Validation du format du numéro étudiant (1 lettre + 3 chiffres)
- Validation de la date de naissance, téléphone et email

3.2.3 Classe Enseignant

```
1 public class Enseignant {
2     private int id; // Identifiant unique
3     private String nom; // Nom de l'enseignant
4     private List<SousModule> sousModulesAssignes; // Liste des sous-
modules assignés
5     // Getters et Setters
6 }
```

Listing 3 – Classe Enseignant - Représentation d'un enseignant



```
J Enseignant.java X
modele > J Enseignant.java
4
5 public class Enseignant {
6     private int id;
7     private String nom;
8     private List<SousModule> sousModulesAssignes = new ArrayList<>();
9
10    public Enseignant(int id, String nom) {
11        this.id = id;
12        this.nom = nom;
13    }
14
15    public int getId() { return id; }
16    public String getNom() { return nom; }
17    public List<SousModule> getSousModulesAssignes() { return sousModulesAssignes; }
18    public void setSousModulesAssignes(List<SousModule> list) { this.sousModulesAssignes = list; }
19    public void setNom(String nom) { this.nom = nom; }
20    public void setId(int id) { this.id = id; }
21
22    @Override
23    public String toString() { return nom; }
24 }
```

FIGURE 4 – Classe Enseignant

Fonctionnalités :

- Gestion des enseignants
- Attribution automatique aux sous-modules
- Compte utilisateur automatique lors de la création

3.2.4 Classe Module

```
1 public class Module {
2     private int id;                // Identifiant unique
3     private String nom;            // Nom du module
4     private String description;    // Description du module
5     private List<SousModule> sousModules; // Liste des sous-modules du
6     module                         // Getters et Setters
7 }
```

Listing 4 – Classe Module - Module d'enseignement

```
J Module.java X
modele > J Module.java > ...
1
2
3
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class Module {
8     private int id;
9     private String nom;
10    private String description;
11    private List<SousModule> sousModules = new ArrayList<>();
12
13    public Module(int id, String nom, String description) {
14        this.id = id;
15        this.nom = nom;
16        this.description = description;
17    }
18
19    public int getId() { return id; }
20    public String getNom() { return nom; }
21    public String getDescription() { return description; }
22    public List<SousModule> getSousModules() { return sousModules; }
23
24    public void setId(int id) { this.id = id; }
25    public void setNom(String nom) { this.nom = nom; }
26    public void setDescription(String description) { this.description = description; }
27    public void ajouterSousModule(SousModule sm) { sousModules.add(sm); }
28
29    @Override
30    public String toString() { return nom; }
31 }
```

FIGURE 5 – Classe Module

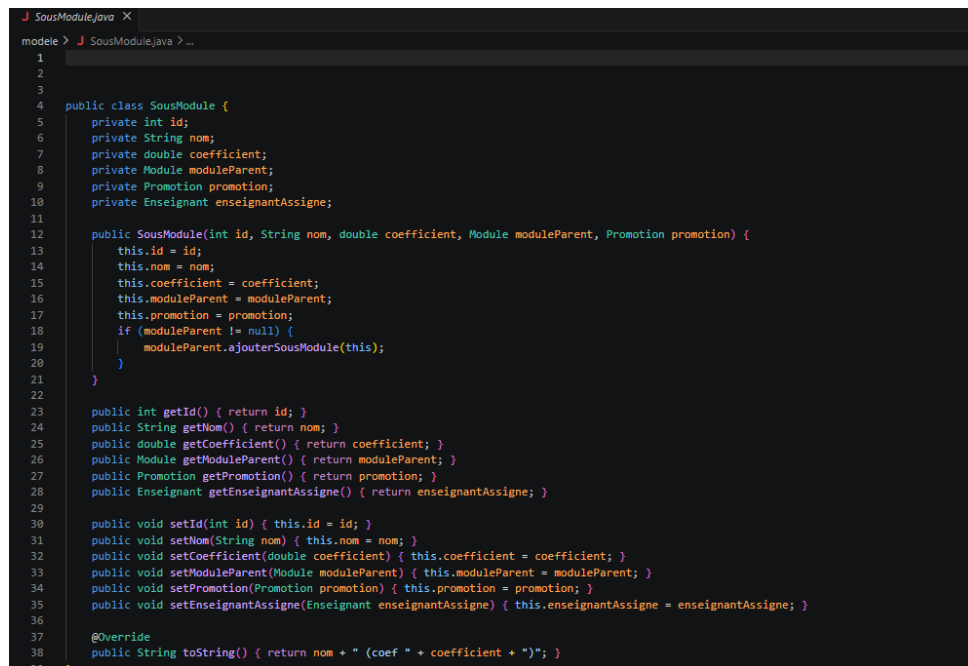
Fonctionnalités :

- Organisation des matières en modules
- Association avec des sous-modules
- Gestion de la description du contenu pédagogique

3.2.5 Classe SousModule

```
1 public class SousModule {
2     private int id;                // Identifiant unique
3     private String nom;            // Nom du sous-module
4     private double coefficient;    // Coefficient pour le calcul des
5                                     moyennes
6     private Module moduleParent;   // Module parent
7     private Promotion promotion;   // Promotion concern e
8     private Enseignant enseignantAssigne; // Enseignant responsable
9     // Getters et Setters
10 }
```

Listing 5 – Classe SousModule - Sous-module d'enseignement



```

1
2
3
4 public class SousModule {
5     private int id;
6     private String nom;
7     private double coefficient;
8     private Module moduleParent;
9     private Promotion promotion;
10    private Enseignant enseignantAssigne;
11
12    public SousModule(int id, String nom, double coefficient, Module moduleParent, Promotion promotion) {
13        this.id = id;
14        this.nom = nom;
15        this.coefficient = coefficient;
16        this.moduleParent = moduleParent;
17        this.promotion = promotion;
18        if (moduleParent != null) {
19            moduleParent.ajouterSousModule(this);
20        }
21    }
22
23    public int getId() { return id; }
24    public String getNom() { return nom; }
25    public double getCoefficient() { return coefficient; }
26    public Module getModuleParent() { return moduleParent; }
27    public Promotion getPromotion() { return promotion; }
28    public Enseignant getEnseignantAssigne() { return enseignantAssigne; }
29
30    public void setId(int id) { this.id = id; }
31    public void setNom(String nom) { this.nom = nom; }
32    public void setCoefficient(double coefficient) { this.coefficient = coefficient; }
33    public void setModuleParent(Module moduleParent) { this.moduleParent = moduleParent; }
34    public void setPromotion(Promotion promotion) { this.promotion = promotion; }
35    public void setEnseignantAssigne(Enseignant enseignantAssigne) { this.enseignantAssigne = enseignantAssigne; }
36
37    @Override
38    public String toString() { return nom + " (coef " + coefficient + ")"; }
39

```

FIGURE 6 – Classe SousModule

Fonctionnalités :

- Gestion des sous-modules avec coefficients
- Assignment d'un enseignant responsable
- Association à une promotion spécifique
- Liens vers le module parent

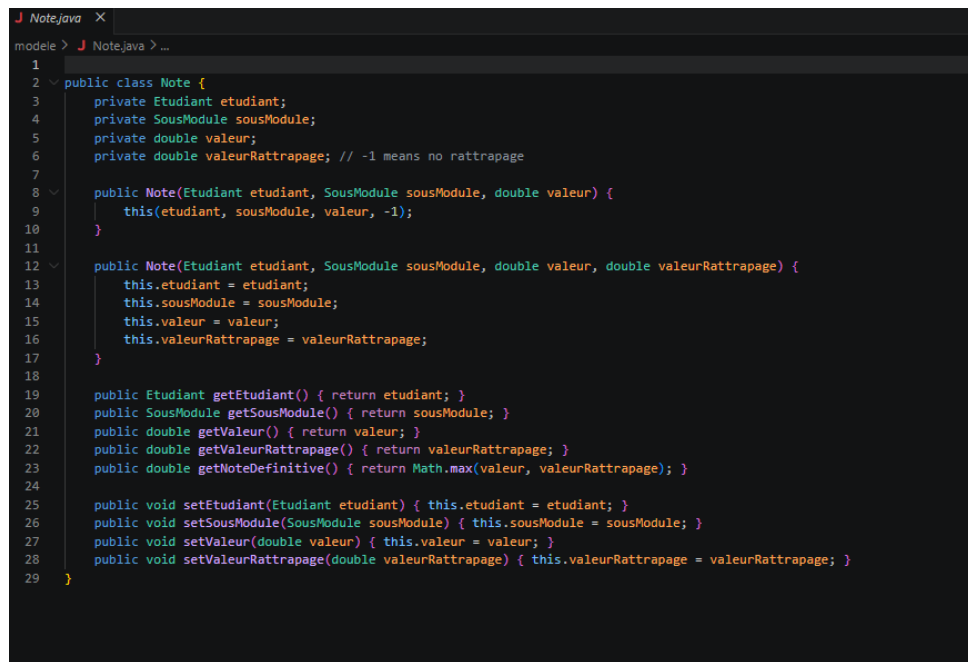
3.2.6 Classe Note

```

1 public class Note {
2     private Etudiant etudiant;           // tudiant concern
3     private SousModule sousModule;       // Sous-module concern
4     private double valeur;               // Note initiale sur 20
5     private double valeurRattrapage;     // Note de rattrapage sur 20 (-1
6     = sans rattrapage)
7     // Getters et Setters
8 }

```

Listing 6 – Classe Note - Gestion des notes



```

1
2 public class Note {
3     private Etudiant etudiant;
4     private SousModule sousModule;
5     private double valeur;
6     private double valeurRattrapage; // -1 means no rattrapage
7
8     public Note(Etudiant etudiant, SousModule sousModule, double valeur) {
9         this(etudiant, sousModule, valeur, -1);
10    }
11
12    public Note(Etudiant etudiant, SousModule sousModule, double valeur, double valeurRattrapage) {
13        this.etudiant = etudiant;
14        this.sousModule = sousModule;
15        this.valeur = valeur;
16        this.valeurRattrapage = valeurRattrapage;
17    }
18
19    public Etudiant getEtudiant() { return etudiant; }
20    public SousModule getSousModule() { return sousModule; }
21    public double getValeur() { return valeur; }
22    public double getValeurRattrapage() { return valeurRattrapage; }
23    public double getNoteDefinitive() { return Math.max(valeur, valeurRattrapage); }
24
25    public void setEtudiant(Etudiant etudiant) { this.etudiant = etudiant; }
26    public void setSousModule(SousModule sousModule) { this.sousModule = sousModule; }
27    public void setValeur(double valeur) { this.valeur = valeur; }
28    public void setValeurRattrapage(double valeurRattrapage) { this.valeurRattrapage = valeurRattrapage; }
29 }

```

FIGURE 7 – Classe Note

Fonctionnalités :

- Gestion des notes initiales et de rattrapage
- Calcul automatique de la note définitive (max entre initiale et rattrapage)
- Contrôle des valeurs (0 à 20)
- Association unique par étudiant/sous-module

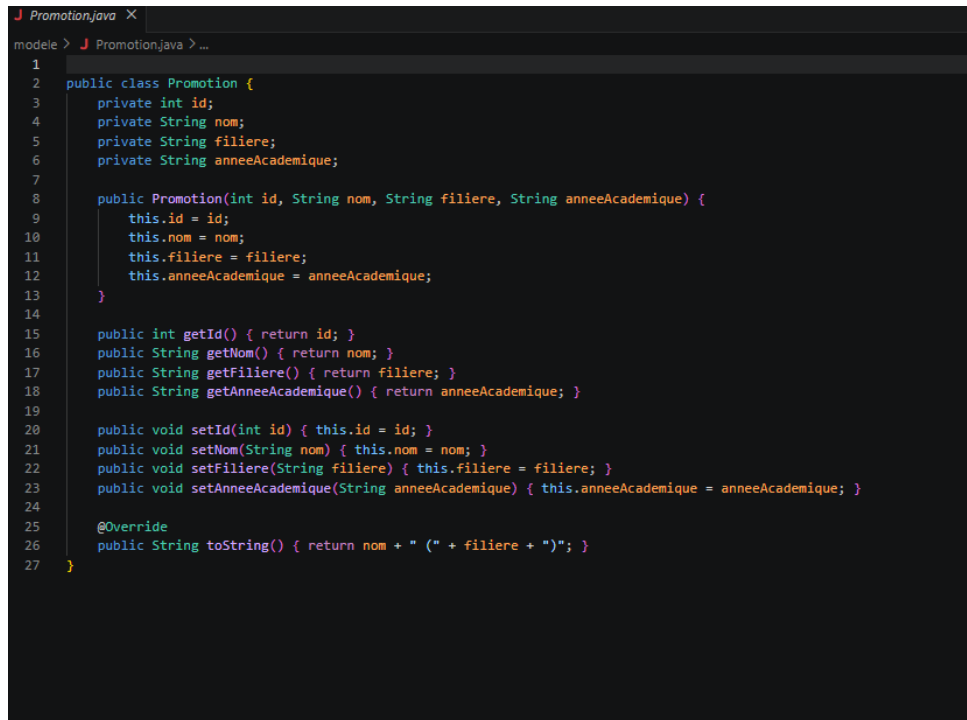
3.2.7 Classe Promotion

```

1 public class Promotion {
2     private int id;                // Identifiant unique
3     private String nom;            // Nom de la promotion (ex: GI1)
4     private String filiere;        // Filière (ex: Génie
Informatique)
5     private String anneeAcademique; // Année académique (ex:
2024/2025)
6     // Getters et Setters
7 }

```

Listing 7 – Classe Promotion - Gestion des promotions

A screenshot of a Java IDE window titled 'Promotion.java'. The code defines a 'Promotion' class with four private attributes: 'id' (int), 'nom' (String), 'filiere' (String), and 'anneeAcademique' (String). It includes a constructor that initializes these attributes, four getter methods, four setter methods, and an overridden 'toString()' method that returns the object's details in a formatted string. The code is as follows:

```
1
2 public class Promotion {
3     private int id;
4     private String nom;
5     private String filiere;
6     private String anneeAcademique;
7
8     public Promotion(int id, String nom, String filiere, String anneeAcademique) {
9         this.id = id;
10        this.nom = nom;
11        this.filiere = filiere;
12        this.anneeAcademique = anneeAcademique;
13    }
14
15    public int getId() { return id; }
16    public String getNom() { return nom; }
17    public String getFiliere() { return filiere; }
18    public String getAnneeAcademique() { return anneeAcademique; }
19
20    public void setId(int id) { this.id = id; }
21    public void setNom(String nom) { this.nom = nom; }
22    public void setFiliere(String filiere) { this.filiere = filiere; }
23    public void setAnneeAcademique(String anneeAcademique) { this.anneeAcademique = anneeAcademique; }
24
25    @Override
26    public String toString() { return nom + " (" + filiere + ")"; }
27 }
```

FIGURE 8 – Classe Promotion

Fonctionnalités :

- Organisation des étudiants par promotion
- Filtrage par filière
- Gestion de l'année académique

4 Structure de la base de données

4.1 Schéma de la base de données

La base de données `gestion_notes` est composée des tables suivantes :

TABLE 2 – Tables de la base de données

Table	Description
utilisateur	Gestion des comptes utilisateurs (nom, mot de passe, rôle)
promotion	Gestion des promotions (nom, filière, année académique)
etudiant	Informations des étudiants (nom, prénom, numéro, archive, etc.)
enseignant	Informations des enseignants
module	Gestion des modules (nom, description)
sous_module	Gestion des sous-modules (nom, coefficient, assignations)
note	Stockage des notes (valeur, valeur_rattrapage)
validation_promotion	Validation des années (valide/ non valide)

4.2 Relations entre les tables

utilisateur	←	etudiant (login)
utilisateur	←	enseignant (login)
promotion	↔	etudiant (promotion_id)
promotion	↔	sous_module (promotion_id)
module	↔	sous_module (module_id)
enseignant	↔	sous_module (enseignant_id)
etudiant	↔	note (etudiant_id)
sous_module	↔	note (sous_module_id)
etudiant	↔	validation_promotion (etudiant_id)
promotion	↔	validation_promotion (promotion_id)

FIGURE 9 – Relations entre les tables

4.3 Requêtes principales

4.3.1 Authentification

Listing 8 – Requête d'authentification

```
SELECT * FROM utilisateur
WHERE nom = ? AND mot_de_passe = ?
```

4.3.2 Récupération des étudiants par promotion

Listing 9 – Récupération des étudiants d’une promotion

```
SELECT * FROM etudiant  
WHERE promotion_id = ? AND archive = false
```

4.3.3 Récupération des notes d’un étudiant

Listing 10 – Récupération des notes d’un étudiant

```
SELECT * FROM note WHERE etudiant_id = ?
```

4.3.4 Insertion/Mise à jour d’une note

Listing 11 – Sauvegarde d’une note

```
INSERT INTO note (etudiant_id, sous_module_id, valeur, valeur_rattrapage)  
VALUES (?, ?, ?, ?)  
ON DUPLICATE KEY UPDATE valeur=?, valeur_rattrapage=?
```



```

        sommeCoef += sm.getCoefficient();
    }
}
return sommeCoef > 0 ? sommeNotes / sommeCoef : -1;
}

// Calcul moyenne sous-module
public static double calculerMoyenneSousModule(SousModule sm, Etudiant etudiant, StockageMySQL db) {
    Note note = trouverNote(etudiant, sm, db);
    if (note == null) return -1;
    return note.getNoteDefinitive();
}

// Calcul moyenne générale
public static double calculerMoyenneGenerale(List<Module> modules, Etudiant etudiant, StockageMySQL db) {
    double sommeNotes = 0;
    int nbModules = 0;
    for (Module m : modules) {
        double moyModule = calculerMoyenneModule(m, etudiant, db);
        if (moyModule >= 0) {
            sommeNotes += moyModule;
            nbModules++;
        }
    }
    return nbModules > 0 ? sommeNotes / nbModules : -1;
}

// Validation année
public static ResultatValidation validerAnnee(Etudiant etudiant, List<Module> modules, StockageMySQL db) {
    double moyenneGenerale = calculerMoyenneGenerale(modules, etudiant, db);
    int modulesNonValides = 0;
    boolean noteModuleInferieure6 = false;

    for (Module m : modules) {
        if (!estModuleValide(m, etudiant, db)) {
            modulesNonValides++;
        }
        double moyModule = calculerMoyenneModule(m, etudiant, db);
        if (moyModule < 6 && moyModule >= 0) {
            noteModuleInferieure6 = true;
        }
    }

    boolean valide = (moyenneGenerale >= 11) && (modulesNonValides <= 2) && !noteModuleInferieure6;
}

```

FIGURE 11 – Suite du calcul des moyennes

```

    return new ResultatValidation(valide, moyenneGenerale, modulesNonValides, noteModuleInferieure6);
}

// Trouver note
private static Note trouverNote(Etudiant etudiant, SousModule sm, StockageMySQL db) {
    for (Note n : db.getNotesByEtudiant(etudiant.getId())) {
        if (n.getSousModule().getId() == sm.getId()) {
            return n;
        }
    }
    return null;
}

// Classe résultat
public static class ResultatValidation {
    public boolean valide;
    public double moyenneGenerale;
    public int modulesNonValides;
    public boolean noteModuleInferieure6;

    public ResultatValidation(boolean valide, double moyenneGenerale, int modulesNonValides, boolean noteModuleInferieure6) {
        this.valide = valide;
        this.moyenneGenerale = moyenneGenerale;
        this.modulesNonValides = modulesNonValides;
        this.noteModuleInferieure6 = noteModuleInferieure6;
    }
}

```

FIGURE 12 – Validation et classe de résultat

5.3 Algorithmes clés

5.3.1 Validation d'un module

```
1 public static boolean estModuleValide(Module module, Etudiant etudiant,
2   StockageMySQL db) {
3   // 1. Calculer la moyenne du module
4   double moyenneModule = calculerMoyenneModule(module, etudiant, db);
5   if (moyenneModule < 11) return false;
6
7   // 2. Vérifier chaque sous-module
8   for (SousModule sm : getSousModules(module, db)) {
9     Note note = trouverNote(etudiant, sm, db);
10    // Note absente ou < 6 -> module non valid
11    if (note != null && note.getNoteDefinitive() < 6) return false;
12    if (note == null) return false;
13  }
14  return true;
15 }
```

Listing 12 – Validation d'un module

5.3.2 Calcul de la moyenne d'un module

```
1 public static double calculerMoyenneModule(Module module, Etudiant
2   etudiant, StockageMySQL db) {
3   double sommeNotes = 0;
4   double sommeCoef = 0;
5   for (SousModule sm : getSousModules(module, db)) {
6     double note = calculerMoyenneSousModule(sm, etudiant, db);
7     if (note >= 0) {
8       sommeNotes += note * sm.getCoefficient();
9       sommeCoef += sm.getCoefficient();
10    }
11  }
12  return sommeCoef > 0 ? sommeNotes / sommeCoef : -1;
13 }
```

Listing 13 – Calcul de la moyenne d'un module

5.3.3 Validation de l'année

```
1 public static ResultatValidation validerAnnee(Etudiant etudiant, List<
2   Module> modules, StockageMySQL db) {
3   double moyenneGenerale = calculerMoyenneGenerale(modules, etudiant,
4     db);
5   int modulesNonValides = 0;
6   boolean noteModuleInferieure6 = false;
7
8   for (Module m : modules) {
9     if (!estModuleValide(m, etudiant, db)) {
10      modulesNonValides++;
11    }
12    double moyModule = calculerMoyenneModule(m, etudiant, db);
13    if (moyModule < 6 && moyModule >= 0) {
```

```
12         noteModuleInferieure6 = true;
13     }
14 }
15
16 boolean valide = (moyenneGenerale >= 11) && (modulesNonValides <= 2)
17 && !noteModuleInferieure6;
18 return new ResultatValidation(valide, moyenneGenerale,
modulesNonValides, noteModuleInferieure6);
18 }
```

Listing 14 – Validation de l'année

6 Interface utilisateur

6.1 Page de connexion

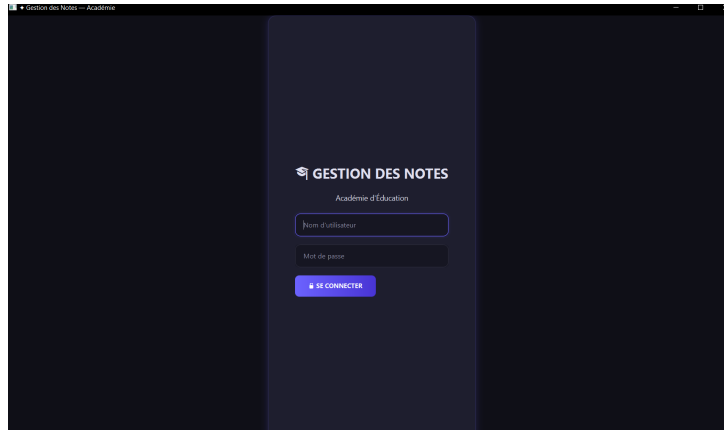


FIGURE 13 – Interface de connexion

Fonctionnalités :

- Saisie du nom d'utilisateur
- Saisie du mot de passe (masqué)
- Bouton "SE CONNECTER" pour l'authentification
- Interface moderne avec thème sombre
- Gestion des erreurs (champs vides, identifiants incorrects)

Authentification :

1. L'utilisateur entre son nom et son mot de passe
2. La méthode `authentifier()` de `StockageMySQL` vérifie en base de données
3. Le rôle de l'utilisateur est récupéré
4. L'interface principale est chargée avec les permissions appropriées

6.2 Interface principale

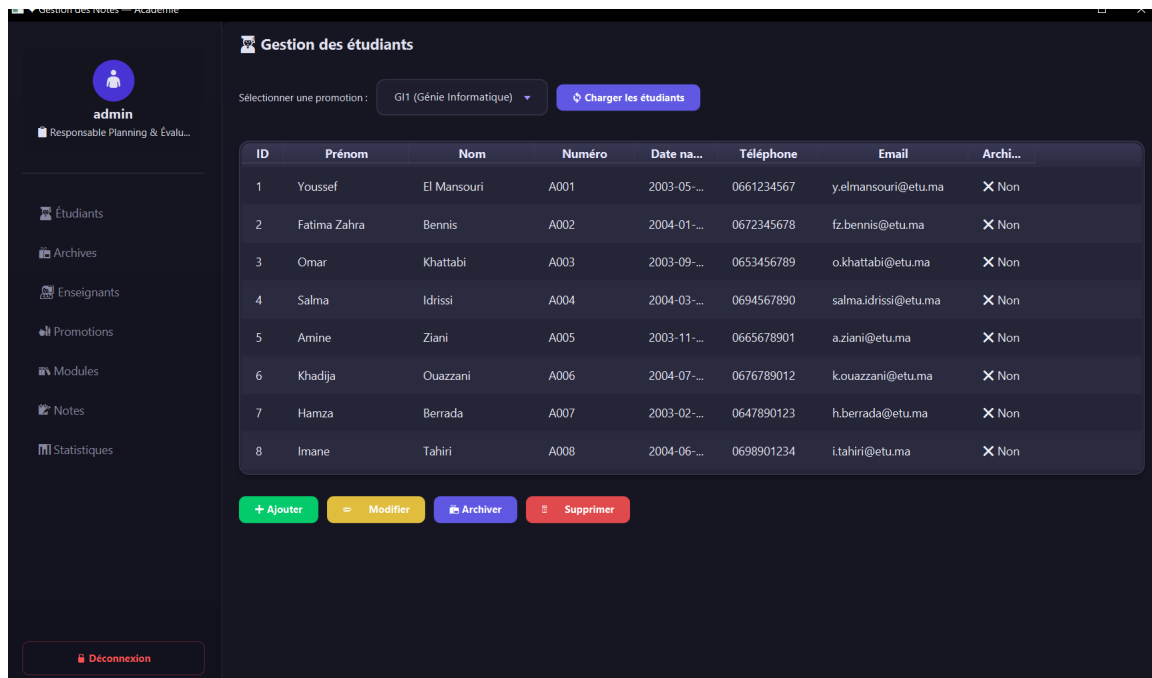


FIGURE 14 – Interface principale - Gestion des étudiants

Structure de l'interface :

1. Barre latérale gauche :

- Profil utilisateur avec avatar
- Nom et rôle de l'utilisateur connecté
- Menu de navigation selon le rôle
- Bouton de déconnexion

2. Panneau central :

- Zone de contenu dynamique
- Affichage des données sous forme de tableaux
- Boutons d'action

6.3 Gestion des étudiants

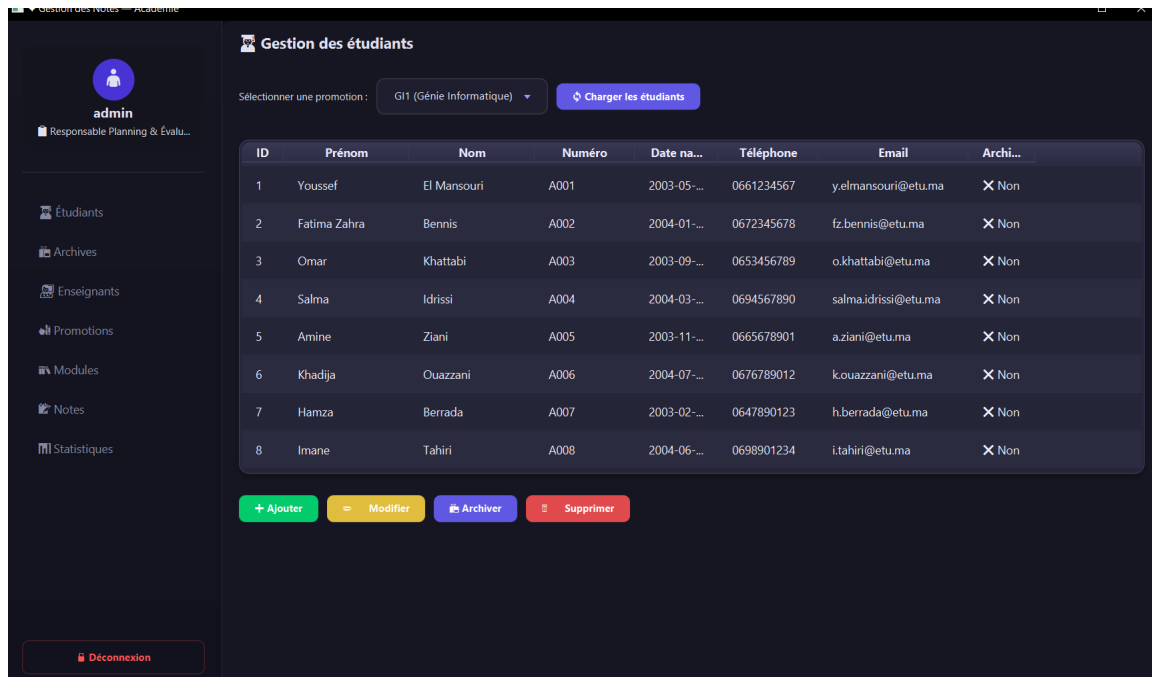


FIGURE 15 – Gestion des étudiants

Fonctionnalités :

- **Sélection de promotion** : Filtrage des étudiants par promotion
- **Chargement** : Affichage des étudiants de la promotion sélectionnée
- **Tableau** : Affichage des informations (ID, Prénom, Nom, Numéro, Date naissance, Téléphone, Email, Archivé)
- **Ajouter** : Création d'un nouvel étudiant avec validation des données
- **Modifier** : Modification des informations d'un étudiant
- **Archiver** : Archivage d'un étudiant (ne plus l'afficher)
- **Supprimer** : Suppression définitive d'un étudiant (avec ses notes et son compte)

Validation des données :

- Numéro étudiant : 1 lettre majuscule + 3 chiffres (ex : A123)
- Date de naissance : DD/MM/YYYY (ex : 15/05/2000)
- Téléphone : 06XXXXXXXX ou 07XXXXXXXX (format marocain)
- Email : format standard (ex : nom@domaine.com)

Création automatique du compte utilisateur :

```

1 // Nom d'utilisateur = prenom + nom (ex: AliceMartin)
2 String nomUtilisateur = et.getPrenom() + et.getNom();
3 // Mot de passe = nom + prenom + dateNaissance (ex: martinalice15052000)
4 String mdpUtilisateur = et.getNom().toLowerCase() + et.getPrenom().
    toLowerCase() + et.getDateNaissance().replace("/", "");
5 db.creerUtilisateurEtudiant(nomUtilisateur, mdpUtilisateur, et.getPrenom()
    + " " + et.getNom());

```

Listing 15 – Création du compte utilisateur

6.4 Gestion des enseignants

Fonctionnalités :

- Affichage de la liste des enseignants
- Ajout d'un enseignant avec création automatique du compte utilisateur
- Modification d'un enseignant (met à jour le compte utilisateur)
- Suppression d'un enseignant (supprime également le compte utilisateur)
- Visualisation du nom d'utilisateur associé

Création automatique du compte utilisateur :

- Nom d'utilisateur = nom de l'enseignant sans espaces
- Mot de passe = nom d'utilisateur + "123"

6.5 Gestion des promotions

Fonctionnalités :

- Affichage des promotions (ID, Nom, Filière, Année académique)
- Ajout d'une nouvelle promotion
- Modification d'une promotion existante
- Suppression d'une promotion
- Filtrage par filière pour les responsables

6.6 Gestion des modules

— Modules :

- Affichage des modules (ID, Nom, Description)
- Ajout, modification et suppression de modules

— Sous-modules :

- Affichage des sous-modules (ID, Nom, Coefficient, Module parent, Promotion, Enseignant)
- Ajout d'un sous-module avec assignation d'enseignant
- Modification d'un sous-module
- Suppression d'un sous-module

6.7 Saisie des notes (Enseignant)

Fonctionnalités :

- Sélection d'un sous-module assigné à l'enseignant
- Chargement des étudiants de la promotion associée
- Saisie directe dans le tableau (édition en ligne)
- Gestion des notes initiales et de rattrapage
- Indicateur de modifications non sauvegardées (lampe rouge/verte)
- Sauvegarde des notes en base de données

- Import des notes depuis Excel
- Export des notes vers Excel

Règles de saisie :

- Note entre 0 et 20
- Si note initiale ≥ 11 , rattrapage non autorisé
- Si note initiale < 11 , rattrapage autorisé
- La note définitive est le max de l'initiale et du rattrapage

Import Excel :

- Format attendu : Nom étudiant | Note initiale | Rattrapage
- Les notes sont automatiquement sauvegardées

Export Excel :

- Génération d'un fichier Excel avec les notes
- Format : Nom étudiant | Note initiale | Rattrapage

6.8 Consultation des notes (Étudiant)

Fonctionnalités :

- Affichage des informations personnelles
- Tableau des notes par module et sous-module
- Affichage des coefficients
- Calcul de la moyenne par module
- Calcul de la moyenne générale
- Affichage du statut (validé/non validé)

6.9 Validation de l'année (Responsable)

- Calcul automatique de la validation selon le règlement
- Affichage des critères :
 - Moyenne générale
 - Nombre de modules non validés
 - Présence de notes < 6
- Validation manuelle (décision du jury)
- Invalidation manuelle

FIGURE 16 – Fonctionnalités de validation

6.10 Statistiques (Planning et Responsable)

Fonctionnalités :

- **Meilleur étudiant** : Affichage du meilleur étudiant de la promotion
- **Classement** : Classement des étudiants par moyenne générale

- **Taux de réussite** : Graphique circulaire des validés/non validés
- **Répartition des moyennes** : Histogramme des tranches (< 11 , 11-13, 13-15, ≥ 15)
- **Listes** : Étudiants validés et non validés

6.11 Progression (Étudiant)

Fonctionnalités :

- Affichage des moyennes par module
- Statut de validation de chaque module
- Moyenne générale
- Statut de l'année (validée/non validée)
- Détails en cas de non validation

6.12 Structure du menu par rôle

TABLE 3 – Fonctionnalités par rôle

Rôle	Fonctionnalités
PLANNING	Étudiants, Enseignants, Archives, Promotions, Modules, Notes (classe), Statistiques
RESPONSABLE_FILIERE	Étudiants (filtrés par filière), Archives, Validation Année, Statistiques
ENSEIGNANT	Saisie des notes (sous-modules assignés uniquement)
ETUDIANT	Mes notes (consultation), Ma progression (statistiques personnelles)

7 Fonctionnalités détaillées

7.1 Gestion des étudiants

7.1.1 Ajout d'un étudiant

1. Sélectionner une promotion
2. Cliquer sur "Ajouter"
3. Remplir le formulaire (Prénom, Nom, Numéro, Date naissance, Téléphone, Email)
4. Validation des données
5. Création automatique du compte utilisateur
6. Affichage des identifiants de connexion

7.1.2 Modification d'un étudiant

1. Sélectionner un étudiant dans le tableau
2. Cliquer sur "Modifier"
3. Modifier les informations
4. Si le numéro ou la date change, mise à jour du compte utilisateur
5. Nouveaux identifiants affichés

7.1.3 Archivage

1. Sélectionner un étudiant
2. Cliquer sur "Archiver"
3. L'étudiant n'apparaît plus dans la liste active
4. Visible dans le panneau "Archives"

7.1.4 Suppression définitive

1. Sélectionner un étudiant
2. Cliquer sur "Supprimer"
3. Confirmation
4. Suppression des notes, validations, compte utilisateur et de l'étudiant

7.2 Gestion des enseignants

7.2.1 Ajout d'un enseignant

1. Cliquer sur "Ajouter"
2. Saisir le nom complet
3. Création automatique du compte utilisateur
4. Identifiants : nom sans espaces / nom sans espaces + "123"

7.2.2 Modification

1. Sélectionner un enseignant
2. Modifier le nom
3. Mise à jour automatique du compte utilisateur

7.2.3 Suppression

1. Sélectionner un enseignant
2. Confirmation
3. Suppression des assignations, du compte utilisateur et de l'enseignant

7.3 Saisie des notes

7.3.1 Édition en ligne

1. Sélectionner un sous-module
2. Cliquer sur "Charger"
3. Double-cliquer sur une cellule de note
4. Saisir la valeur (0-20)
5. Appuyer sur Entrée
6. Indicateur de modification (lampe rouge)

7.3.2 Sauvegarde

1. Cliquer sur "Enregistrer notes"
2. Sauvegarde de toutes les notes en base de données
3. Lampe passe au vert

7.3.3 Import Excel

1. Cliquer sur "Importer Excel"
2. Sélectionner un fichier Excel
3. Format : Nom étudiant | Note initiale | Rattrapage
4. Import automatique et sauvegarde

7.3.4 Export Excel

1. Cliquer sur "Exporter Excel"
2. Choisir l'emplacement de sauvegarde
3. Génération du fichier avec les notes

7.4 Validation de l'année

7.4.1 Calcul automatique

1. Sélectionner une promotion
2. Cliquer sur "Calculer validation"
3. Affichage des résultats par étudiant
4. Application des règles :
 - Moyenne générale ≥ 11
 - Modules non validés ≤ 2
 - Pas de note < 6

7.4.2 Décision du jury

1. Sélectionner un étudiant
2. Cliquer sur "Valider sélectionné" ou "Invalider sélectionné"
3. Sauvegarde de la décision
4. Mise à jour de l'affichage

7.5 Statistiques

7.5.1 Classement

1. Sélectionner une promotion
2. Cliquer sur "Afficher statistiques"
3. Affichage du classement par moyenne générale
4. Meilleur étudiant mis en avant

7.5.2 Graphiques

- **Taux de réussite** : Diagramme circulaire des validés/non validés
- **Répartition des moyennes** : Histogramme des tranches de moyennes

7.5.3 Bilan

- Liste des étudiants validés avec leurs moyennes
- Liste des étudiants non validés avec leurs moyennes

8 Données initiales de test

8.1 Utilisateurs

TABLE 4 – Utilisateurs de test

id	nom	mot_de_passe	role	filiere
1	admin	admin123	PLANNING	NULL
2	Dr. Bennani	filiere123	RESPONSABLE_FILIERE	Génie Informatique
3	Dr. El Amrani	filiere456	RESPONSABLE_FILIERE	Génie Civil
4	Prof. Alaoui	prof123	ENSEIGNANT	NULL
5	Prof. Tazi	prof123	ENSEIGNANT	NULL
6	Prof. Fassi	prof123	ENSEIGNANT	NULL
7	Prof. Chraibi	prof123	ENSEIGNANT	NULL

8.2 Étudiants

TABLE 5 – Étudiants de test (extrait)

id	prenom	nom	num_etudiant	promotion_id	archive
1	Youssef	El Mansouri	A001	1 (GI1)	0
2	Fatima Zahra	Bennis	A002	1 (GI1)	0
3	Omar	Khattabi	A003	1 (GI1)	0
4	Salma	Idrissi	A004	1 (GI1)	0
5	Amine	Ziani	A005	1 (GI1)	0

8.3 Promotions

TABLE 6 – Promotions de test

id	nom	filiere	annee_academique
1	GI1	Génie Informatique	2024/2025
2	GI2	Génie Informatique	2024/2025
3	GI3	Génie Informatique	2024/2025
4	GC1	Génie Civil	2024/2025

8.4 Modules et Sous-modules

TABLE 7 – Modules de test

id	nom	description
1	Programmation Java	POO, collections, JavaFX
2	Bases de Données	SQL, JDBC, ORM
3	Réseaux Informatiques	TCP/IP, architecture
4	Mathématiques Appliquées	Algèbre, analyse, probabilités

TABLE 8 – Sous-modules de test (extrait)

id	nom	coefficient	module_id	promotion_id	enseignant_id
1	POO Avancée	2.0	1	1	1
2	Collections & Generics	1.0	1	1	1
3	JavaFX & GUI	1.5	1	1	1
4	SQL Avancé	2.0	2	1	2
5	JDBC & ORM	1.0	2	1	2

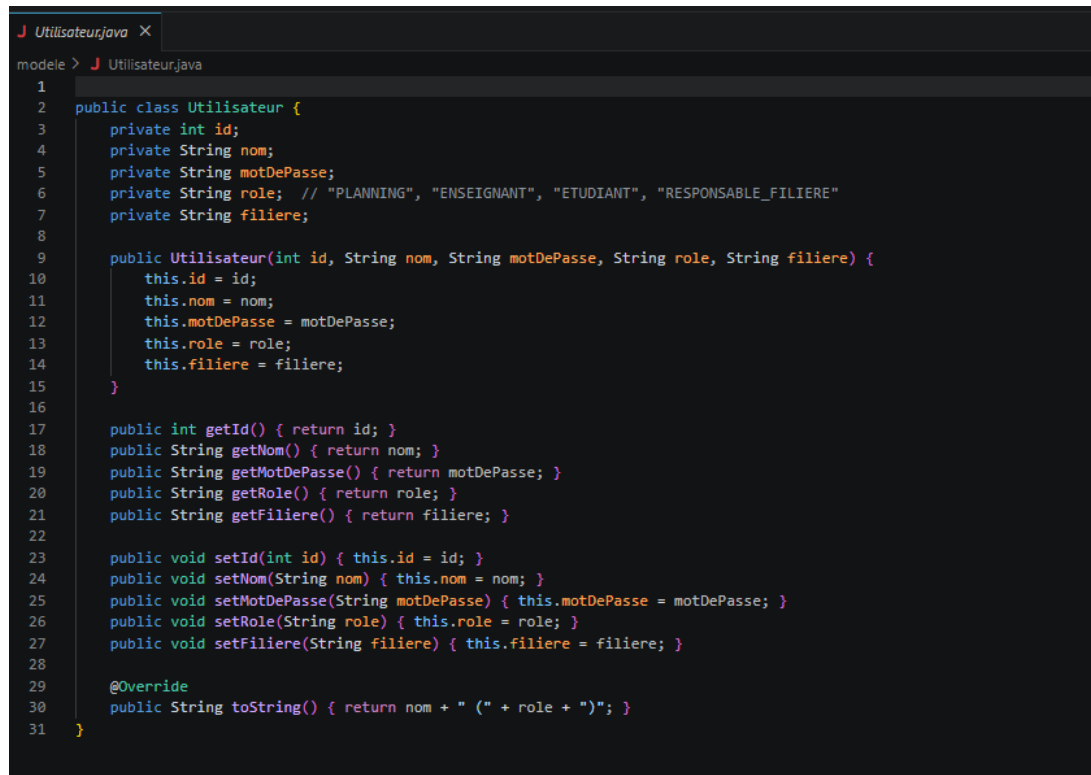
8.5 Notes de test

TABLE 9 – Notes de test (extrait - Étudiant 1)

sous_module_id	valeur	valeur_rattrapage	note définitive
1	16.5	-1	16.5
2	15.0	-1	15.0
3	17.0	-1	17.0
4	14.5	-1	14.5
5	13.0	-1	13.0

9 Description détaillée des captures d'écran

9.1 Capture 1 : Classe Utilisateur



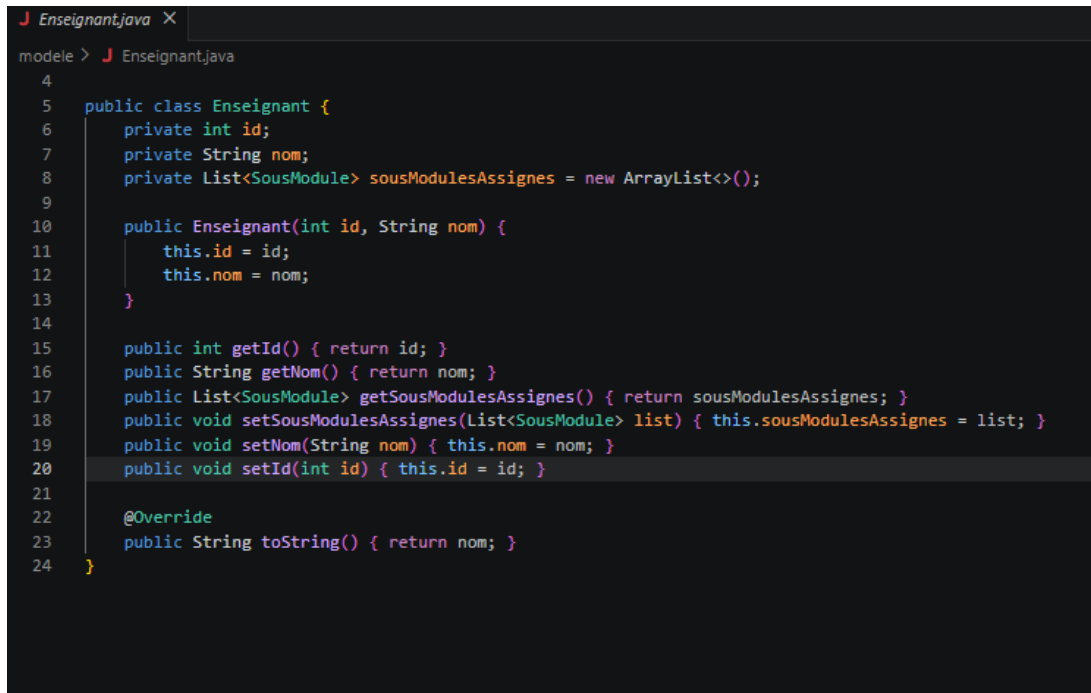
```
1
2 public class Utilisateur {
3     private int id;
4     private String nom;
5     private String motDePasse;
6     private String role; // "PLANNING", "ENSEIGNANT", "ETUDIANT", "RESPONSABLE_FILIERE"
7     private String filiere;
8
9     public Utilisateur(int id, String nom, String motDePasse, String role, String filiere) {
10         this.id = id;
11         this.nom = nom;
12         this.motDePasse = motDePasse;
13         this.role = role;
14         this.filiere = filiere;
15     }
16
17     public int getId() { return id; }
18     public String getNom() { return nom; }
19     public String getMotDePasse() { return motDePasse; }
20     public String getRole() { return role; }
21     public String getFiliere() { return filiere; }
22
23     public void setId(int id) { this.id = id; }
24     public void setNom(String nom) { this.nom = nom; }
25     public void setMotDePasse(String motDePasse) { this.motDePasse = motDePasse; }
26     public void setRole(String role) { this.role = role; }
27     public void setFiliere(String filiere) { this.filiere = filiere; }
28
29     @Override
30     public String toString() { return nom + " (" + role + ")"; }
31 }
```

FIGURE 17 – Classe Utilisateur - Gestion des comptes

Description : La classe `Utilisateur` gère les comptes utilisateurs du système. Elle contient :

- `id` : Identifiant unique généré par la base de données
- `nom` : Nom d'utilisateur utilisé pour la connexion
- `motDePasse` : Mot de passe (stocké en clair dans cette version)
- `role` : Rôle de l'utilisateur (PLANNING, ENSEIGNANT, ETUDIANT, RESPONSABLE_FILIERE)
- `filiere` : Filière pour les responsables de filière

9.2 Capture 2 : Classe Enseignant



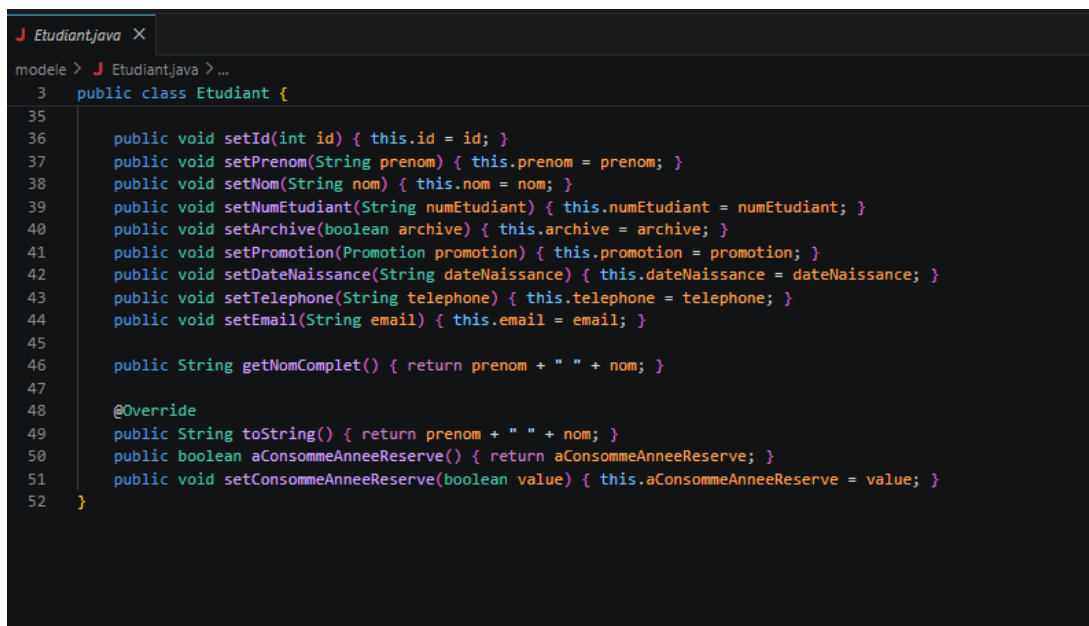
```
J Enseignant.java X
modele > J Enseignant.java
4
5 public class Enseignant {
6     private int id;
7     private String nom;
8     private List<SousModule> sousModulesAssignes = new ArrayList<>();
9
10    public Enseignant(int id, String nom) {
11        this.id = id;
12        this.nom = nom;
13    }
14
15    public int getId() { return id; }
16    public String getNom() { return nom; }
17    public List<SousModule> getSousModulesAssignes() { return sousModulesAssignes; }
18    public void setSousModulesAssignes(List<SousModule> list) { this.sousModulesAssignes = list; }
19    public void setNom(String nom) { this.nom = nom; }
20    public void setId(int id) { this.id = id; }
21
22    @Override
23    public String toString() { return nom; }
24 }
```

FIGURE 18 – Classe Enseignant

Description : La classe `Enseignant` représente un enseignant avec :

- `id` : Identifiant unique
- `nom` : Nom de l'enseignant
- `sousModulesAssignes` : Liste des sous-modules qui lui sont assignés

9.3 Capture 3 : Classe Etudiant



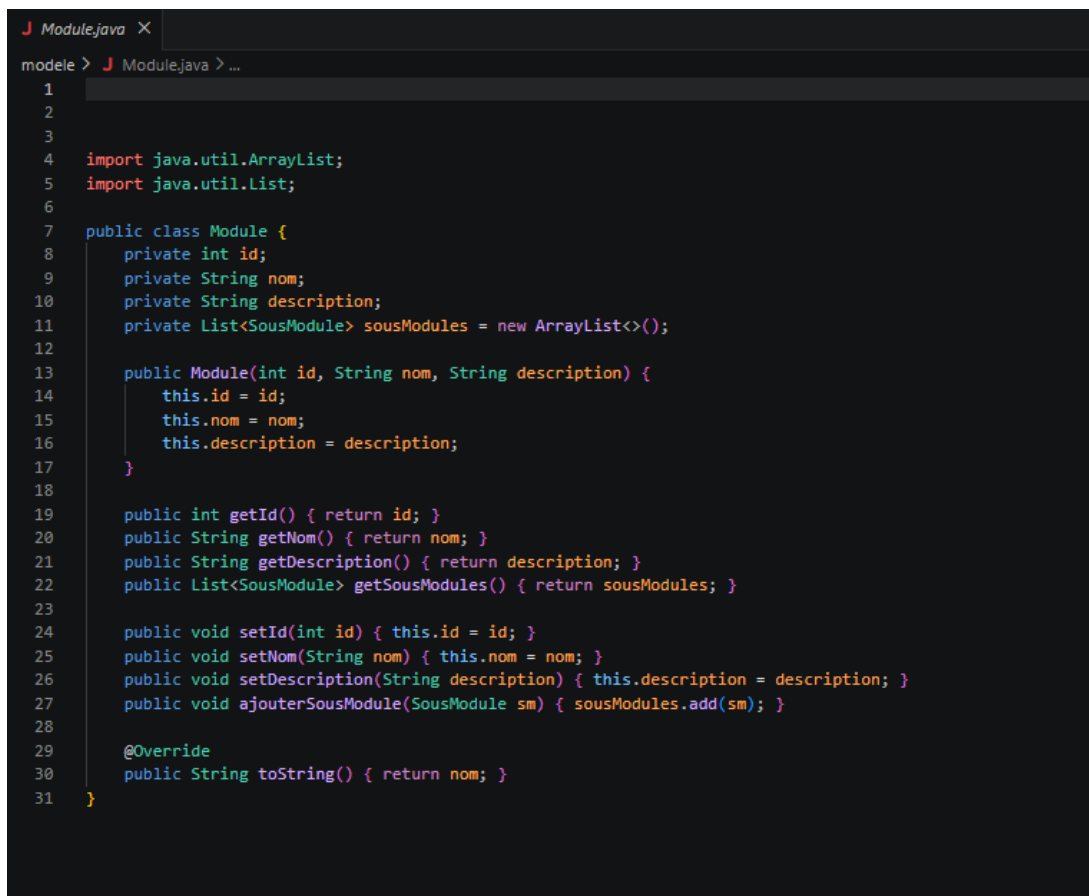
```
J Etudiant.java X
modele > J Etudiant.java > ...
3 public class Etudiant {
35
36     public void setId(int id) { this.id = id; }
37     public void setPrenom(String prenom) { this.prenom = prenom; }
38     public void setNom(String nom) { this.nom = nom; }
39     public void setNumEtudiant(String numEtudiant) { this.numEtudiant = numEtudiant; }
40     public void setArchive(boolean archive) { this.archive = archive; }
41     public void setPromotion(Promotion promotion) { this.promotion = promotion; }
42     public void setDateNaissance(String dateNaissance) { this.dateNaissance = dateNaissance; }
43     public void setTelephone(String telephone) { this.telephone = telephone; }
44     public void setEmail(String email) { this.email = email; }
45
46     public String getNomComple() { return prenom + " " + nom; }
47
48     @Override
49     public String toString() { return prenom + " " + nom; }
50     public boolean aConsommeAnneeReserve() { return aConsommeAnneeReserve; }
51     public void setConsommeAnneeReserve(boolean value) { this.aConsommeAnneeReserve = value; }
52 }
```

FIGURE 19 – Classe Etudiant

Description : La classe `Etudiant` contient toutes les informations d'un étudiant :

- Informations personnelles : prenom, nom, numEtudiant, dateNaissance, telephone, email
- Informations académiques : promotion, archive, aConsommeAnneeReserve
- Méthodes pour la gestion des données

9.4 Capture 4 : Classe Module



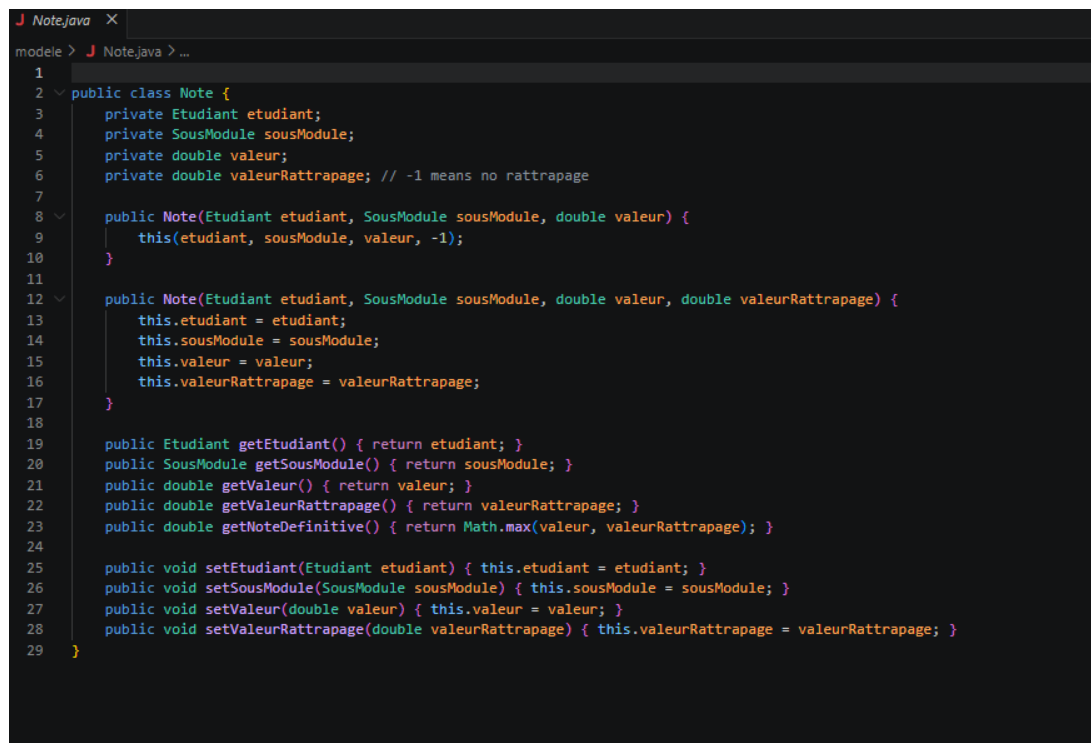
```
1
2
3
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class Module {
8     private int id;
9     private String nom;
10    private String description;
11    private List<SousModule> sousModules = new ArrayList<>();
12
13    public Module(int id, String nom, String description) {
14        this.id = id;
15        this.nom = nom;
16        this.description = description;
17    }
18
19    public int getId() { return id; }
20    public String getNom() { return nom; }
21    public String getDescription() { return description; }
22    public List<SousModule> getSousModules() { return sousModules; }
23
24    public void setId(int id) { this.id = id; }
25    public void setNom(String nom) { this.nom = nom; }
26    public void setDescription(String description) { this.description = description; }
27    public void ajouterSousModule(SousModule sm) { sousModules.add(sm); }
28
29    @Override
30    public String toString() { return nom; }
31 }
```

FIGURE 20 – Classe Module

Description : La classe Module représente un module d'enseignement avec :

- id : Identifiant unique
- nom : Nom du module
- description : Description du contenu
- sousModules : Liste des sous-modules du module

9.5 Capture 5 : Classe Note



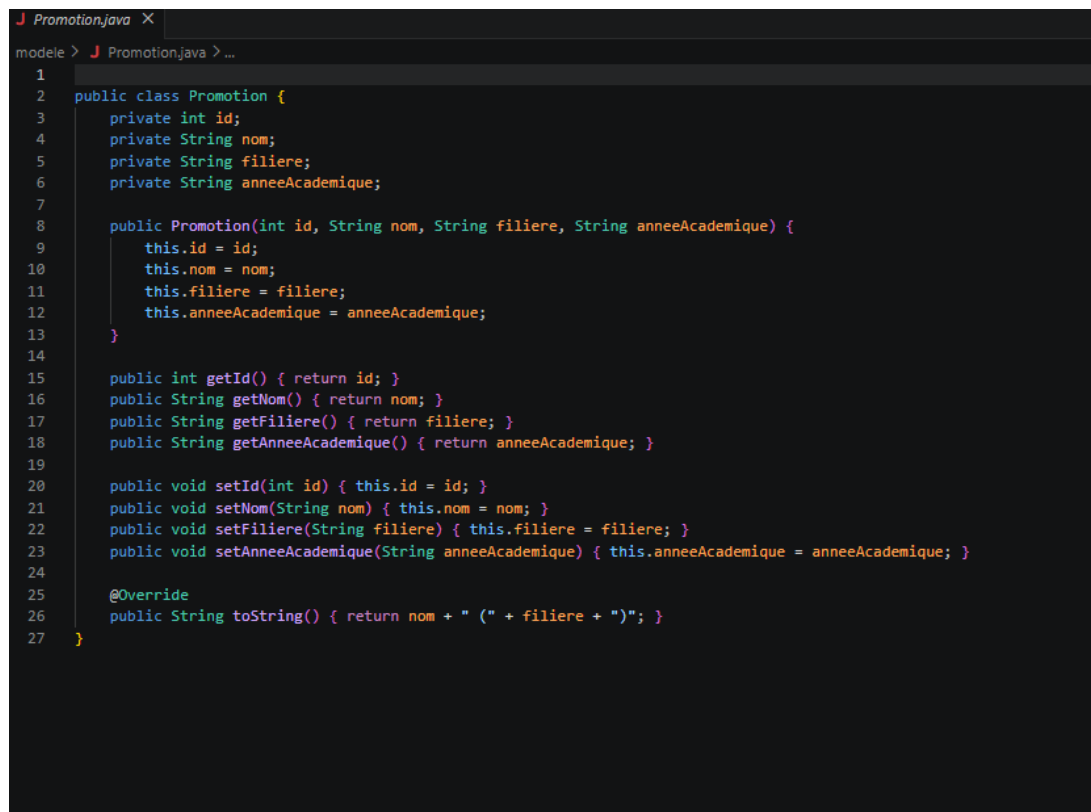
```
1
2 public class Note {
3     private Etudiant etudiant;
4     private SousModule sousModule;
5     private double valeur;
6     private double valeurRattrapage; // -1 means no rattrapage
7
8     public Note(Etudiant etudiant, SousModule sousModule, double valeur) {
9         this(etudiant, sousModule, valeur, -1);
10    }
11
12    public Note(Etudiant etudiant, SousModule sousModule, double valeur, double valeurRattrapage) {
13        this.etudiant = etudiant;
14        this.sousModule = sousModule;
15        this.valeur = valeur;
16        this.valeurRattrapage = valeurRattrapage;
17    }
18
19    public Etudiant getEtudiant() { return etudiant; }
20    public SousModule getSousModule() { return sousModule; }
21    public double getValeur() { return valeur; }
22    public double getValeurRattrapage() { return valeurRattrapage; }
23    public double getNoteDefinitive() { return Math.max(valeur, valeurRattrapage); }
24
25    public void setEtudiant(Etudiant etudiant) { this.etudiant = etudiant; }
26    public void setSousModule(SousModule sousModule) { this.sousModule = sousModule; }
27    public void setValeur(double valeur) { this.valeur = valeur; }
28    public void setValeurRattrapage(double valeurRattrapage) { this.valeurRattrapage = valeurRattrapage; }
29 }
```

FIGURE 21 – Classe Note

Description : La classe Note gère les évaluations :

- `etudiant` : Étudiant concerné
- `sousModule` : Sous-module concerné
- `valeur` : Note initiale sur 20
- `valeurRattrapage` : Note de rattrapage (-1 si non saisi)
- `getNoteDefinitive()` : Retourne le max des deux notes

9.6 Capture 6 : Classe Promotion



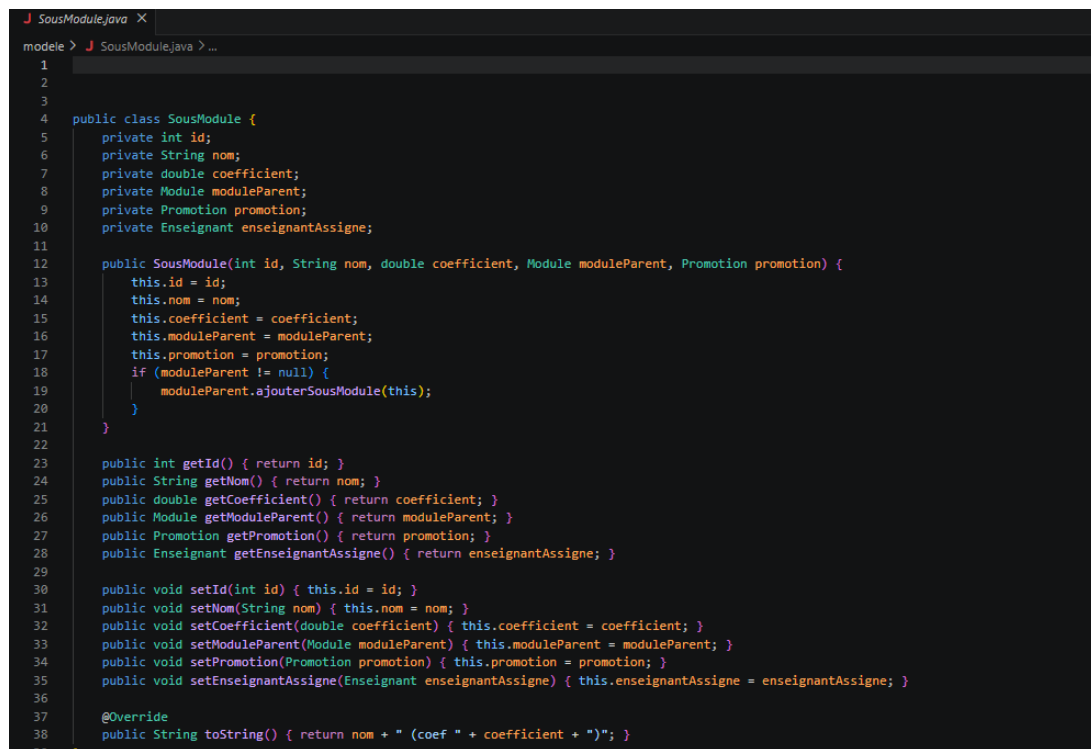
```
1
2 public class Promotion {
3     private int id;
4     private String nom;
5     private String filiere;
6     private String anneeAcademique;
7
8     public Promotion(int id, String nom, String filiere, String anneeAcademique) {
9         this.id = id;
10        this.nom = nom;
11        this.filiere = filiere;
12        this.anneeAcademique = anneeAcademique;
13    }
14
15    public int getId() { return id; }
16    public String getNom() { return nom; }
17    public String getFiliere() { return filiere; }
18    public String getAnneeAcademique() { return anneeAcademique; }
19
20    public void setId(int id) { this.id = id; }
21    public void setNom(String nom) { this.nom = nom; }
22    public void setFiliere(String filiere) { this.filiere = filiere; }
23    public void setAnneeAcademique(String anneeAcademique) { this.anneeAcademique = anneeAcademique; }
24
25    @Override
26    public String toString() { return nom + " (" + filiere + ")"; }
27 }
```

FIGURE 22 – Classe Promotion

Description : La classe `Promotion` organise les étudiants :

- `id` : Identifiant unique
- `nom` : Nom de la promotion (ex : GI1)
- `filiere` : Filière associée
- `anneeAcademique` : Année académique

9.7 Capture 7 : Classe SousModule



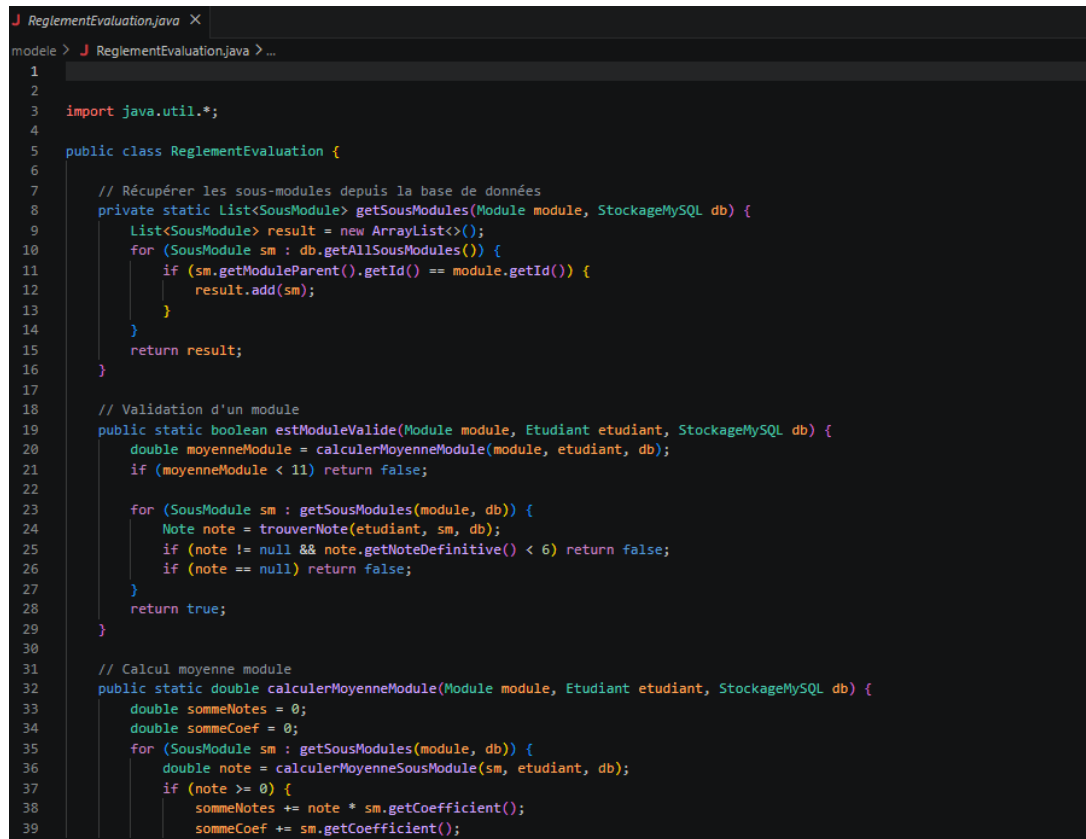
```
1
2
3
4 public class SousModule {
5     private int id;
6     private String nom;
7     private double coefficient;
8     private Module moduleParent;
9     private Promotion promotion;
10    private Enseignant enseignantAssigne;
11
12    public SousModule(int id, String nom, double coefficient, Module moduleParent, Promotion promotion) {
13        this.id = id;
14        this.nom = nom;
15        this.coefficient = coefficient;
16        this.moduleParent = moduleParent;
17        this.promotion = promotion;
18        if (moduleParent != null) {
19            moduleParent.ajouterSousModule(this);
20        }
21    }
22
23    public int getId() { return id; }
24    public String getNom() { return nom; }
25    public double getCoefficient() { return coefficient; }
26    public Module getModuleParent() { return moduleParent; }
27    public Promotion getPromotion() { return promotion; }
28    public Enseignant getEnseignantAssigne() { return enseignantAssigne; }
29
30    public void setId(int id) { this.id = id; }
31    public void setNom(String nom) { this.nom = nom; }
32    public void setCoefficient(double coefficient) { this.coefficient = coefficient; }
33    public void setModuleParent(Module moduleParent) { this.moduleParent = moduleParent; }
34    public void setPromotion(Promotion promotion) { this.promotion = promotion; }
35    public void setEnseignantAssigne(Enseignant enseignantAssigne) { this.enseignantAssigne = enseignantAssigne; }
36
37    @Override
38    public String toString() { return nom + " (coef " + coefficient + ")"; }
39 }
```

FIGURE 23 – Classe SousModule

Description : La classe `SousModule` représente une unité d'enseignement :

- `id` : Identifiant unique
- `nom` : Nom du sous-module
- `coefficient` : Coefficient pour le calcul de la moyenne
- `moduleParent` : Module parent
- `promotion` : Promotion concernée
- `enseignantAssigne` : Enseignant responsable

9.8 Captures 8, 9, 10 : ReglementEvaluation



```
1  ReglementEvaluation.java X
2
3  import java.util.*;
4
5  public class ReglementEvaluation {
6
7      // Récupérer les sous-modules depuis la base de données
8      private static List<SousModule> getSousModules(Module module, StockageMySQL db) {
9          List<SousModule> result = new ArrayList<>();
10         for (SousModule sm : db.getAllSousModules()) {
11             if (sm.getModuleParent().getId() == module.getId()) {
12                 result.add(sm);
13             }
14         }
15         return result;
16     }
17
18     // Validation d'un module
19     public static boolean estModuleValide(Module module, Etudiant etudiant, StockageMySQL db) {
20         double moyenneModule = calculerMoyenneModule(module, etudiant, db);
21         if (moyenneModule < 11) return false;
22
23         for (SousModule sm : getSousModules(module, db)) {
24             Note note = trouverNote(etudiant, sm, db);
25             if (note != null && note.getNoteDefinitive() < 6) return false;
26             if (note == null) return false;
27         }
28         return true;
29     }
30
31     // Calcul moyenne module
32     public static double calculerMoyenneModule(Module module, Etudiant etudiant, StockageMySQL db) {
33         double sommeNotes = 0;
34         double sommeCoef = 0;
35         for (SousModule sm : getSousModules(module, db)) {
36             double note = calculerMoyenneSousModule(sm, etudiant, db);
37             if (note >= 0) {
38                 sommeNotes += note * sm.getCoefficient();
39                 sommeCoef += sm.getCoefficient();
40             }
41         }
42         return sommeNotes / sommeCoef;
43     }
44 }
```

FIGURE 24 – ReglementEvaluation - Partie 1

```

        sommeCoef += sm.getCoefficient();
    }
}
return sommeCoef > 0 ? sommeNotes / sommeCoef : -1;
}

// Calcul moyenne sous-module
public static double calculerMoyenneSousModule(SousModule sm, Etudiant etudiant, StockageMySQL db) {
    Note note = trouverNote(etudiant, sm, db);
    if (note == null) return -1;
    return note.getNoteDefinitive();
}

// Calcul moyenne générale
public static double calculerMoyenneGenerale(List<Module> modules, Etudiant etudiant, StockageMySQL db) {
    double sommeNotes = 0;
    int nbModules = 0;
    for (Module m : modules) {
        double moyModule = calculerMoyenneModule(m, etudiant, db);
        if (moyModule >= 0) {
            sommeNotes += moyModule;
            nbModules++;
        }
    }
    return nbModules > 0 ? sommeNotes / nbModules : -1;
}

// Validation année
public static ResultatValidation validerAnnee(Etudiant etudiant, List<Module> modules, StockageMySQL db) {
    double moyenneGenerale = calculerMoyenneGenerale(modules, etudiant, db);
    int modulesNonValides = 0;
    boolean noteModuleInferieure6 = false;

    for (Module m : modules) {
        if (!estModuleValide(m, etudiant, db)) {
            modulesNonValides++;
        }
        double moyModule = calculerMoyenneModule(m, etudiant, db);
        if (moyModule < 6 && moyModule >= 0) {
            noteModuleInferieure6 = true;
        }
    }

    boolean valide = (moyenneGenerale >= 11) && (modulesNonValides <= 2) && !noteModuleInferieure6;
}

```

FIGURE 25 – ReglementEvaluation - Partie 2

```

    return new ResultatValidation(valide, moyenneGenerale, modulesNonValides, noteModuleInferieure6);
}

// Trouver note
private static Note trouverNote(Etudiant etudiant, SousModule sm, StockageMySQL db) {
    for (Note n : db.getNotesByEtudiant(etudiant.getId())) {
        if (n.getSousModule().getId() == sm.getId()) {
            return n;
        }
    }
    return null;
}

// Classe résultat
public static class ResultatValidation {
    public boolean valide;
    public double moyenneGenerale;
    public int modulesNonValides;
    public boolean noteModuleInferieure6;

    public ResultatValidation(boolean valide, double moyenneGenerale, int modulesNonValides, boolean noteModuleInferieure6) {
        this.valide = valide;
        this.moyenneGenerale = moyenneGenerale;
        this.modulesNonValides = modulesNonValides;
        this.noteModuleInferieure6 = noteModuleInferieure6;
    }
}
}

```

FIGURE 26 – ReglementEvaluation - Partie 3

Description : La classe ReglementEvaluation implémente :

— `estModuleValide()` : Vérifie si un module est validé

- `calculerMoyenneModule()` : Calcule la moyenne d'un module
- `calculerMoyenneGenerale()` : Calcule la moyenne générale
- `validerAnnee()` : Valide l'année selon les règles
- `ResultatValidation` : Classe interne pour les résultats

9.9 Capture 11 : Page de connexion

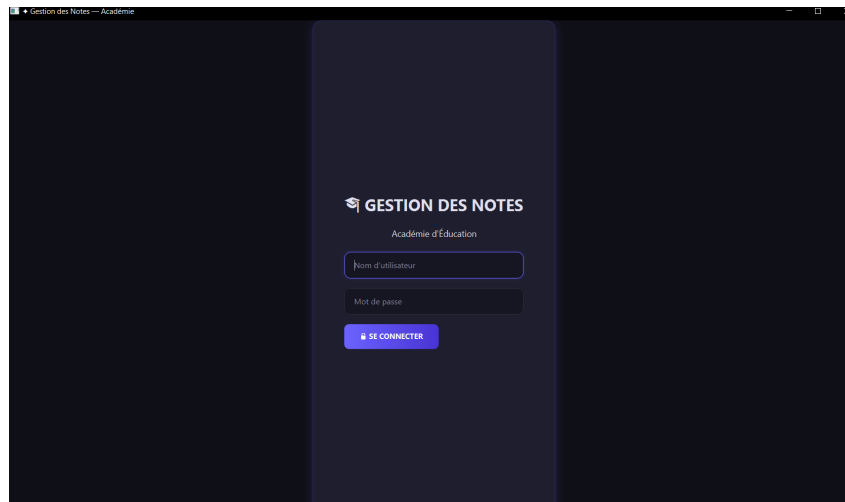


FIGURE 27 – Page de connexion

Description : Interface de connexion avec :

- Logo et titre de l'application
- Champ "Nom d'utilisateur"
- Champ "Mot de passe"
- Bouton "SE CONNECTER"
- Gestion des erreurs (champs vides, identifiants incorrects)

9.10 Capture 12 : Interface principale

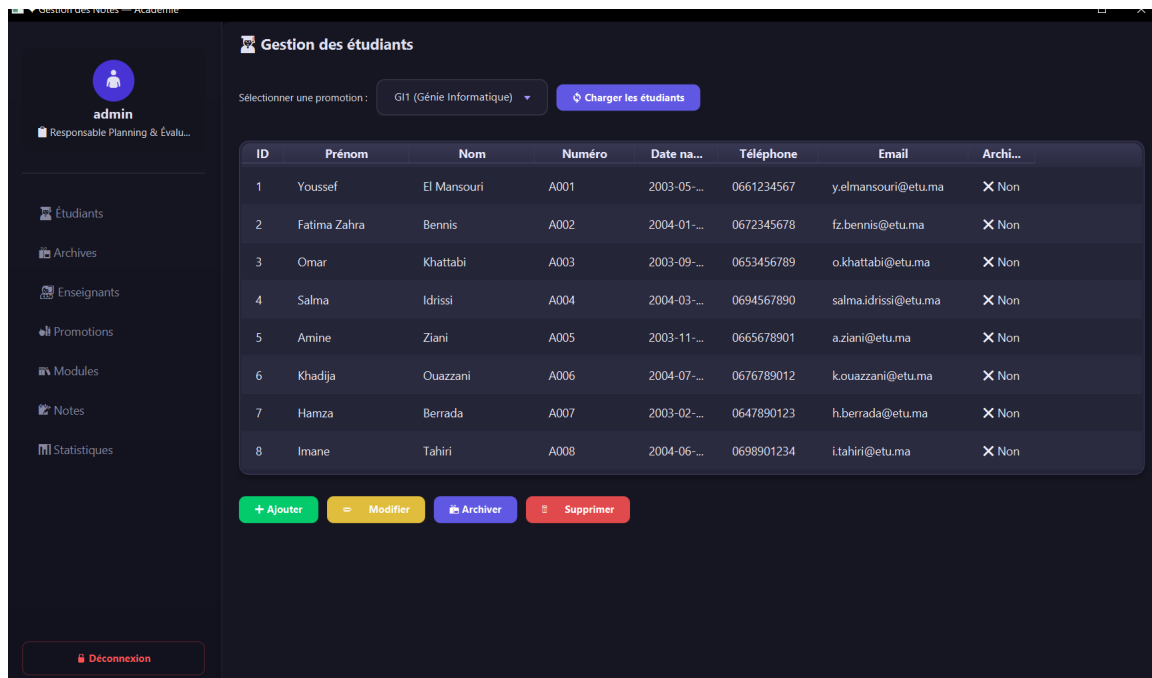


FIGURE 28 – Interface principale - Gestion des étudiants

Description : Interface principale montrant :

- Barre latérale avec profil utilisateur
- Menu de navigation
- Panneau de gestion des étudiants
- Sélecteur de promotion
- Tableau des étudiants
- Boutons d'action (Ajouter, Modifier, Archiver, Supprimer)

9.11 Capture 13 : Statistiques

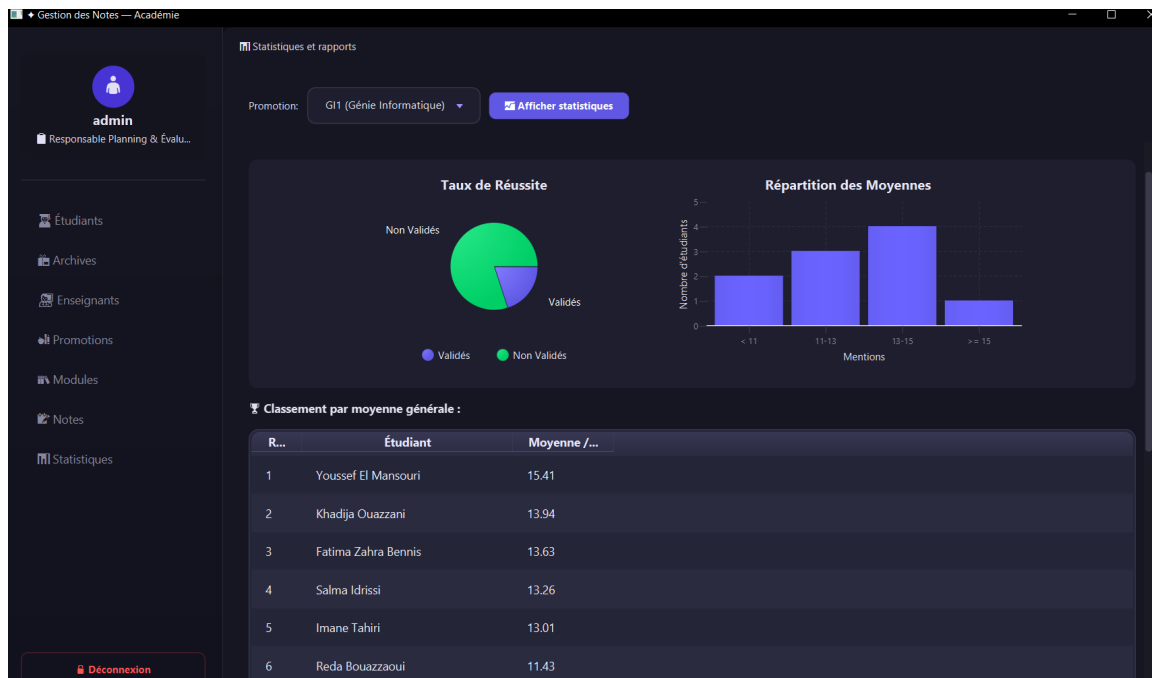


FIGURE 29 – Statistiques et rapports

Description : Panneau de statistiques avec :

- Sélecteur de promotion
- Meilleur étudiant mis en avant
- Classement par moyenne générale
- Taux de réussite (diagramme circulaire)
- Répartition des moyennes (histogramme)
- Listes des étudiants validés et non validés

10 Conclusion

10.1 Bilan du projet

Le projet de gestion des notes a permis de développer une application complète répondant aux besoins des établissements d'enseignement supérieur. Les fonctionnalités implémentées couvrent l'ensemble du cycle de gestion des évaluations.

10.2 Points forts

- **Interface utilisateur** : Moderne, intuitive et responsive
- **Gestion multi-rôles** : Permissions adaptées à chaque type d'utilisateur
- **Système de validation** : Conforme au règlement d'évaluation
- **Import/Export** : Intégration avec Excel via Apache POI
- **Statistiques** : Visualisation complète des données
- **Sécurité** : Système d'authentification robuste

10.3 Fonctionnalités réalisées

- Authentification et gestion des utilisateurs
- Gestion complète des étudiants (CRUD)
- Gestion des enseignants avec création automatique des comptes
- Gestion des promotions, modules et sous-modules
- Saisie des notes avec édition en ligne
- Calcul automatique des moyennes
- Validation selon le règlement d'évaluation
- Statistiques et graphiques
- Import/Export Excel
- Interface adaptée à chaque rôle

10.4 Améliorations possibles

- Hashage des mots de passe (BCrypt ou SHA-256)
- Génération de relevés de notes en PDF
- Notifications par email
- Application mobile
- Interface web avec Spring Boot
- Sauvegarde automatique des notes

10.5 Équipe de développement

Ce projet a été réalisé par :

- **EL MAZYANI MOHAMED ILYAS**
- **HAMZA ABAGAR**
- **BRIK AOUSSAR**

Sous l'encadrement de **Dr. Khammal**.

10.6 Remerciements

Les développeurs remercient :

- **Dr. Khammal** pour son encadrement et ses conseils

11 Annexes

11.1 Configuration du projet

```
1 {  
2     "java.project.sourcePaths": ["modele"],  
3     "java.project.outputPath": "bin",  
4     "java.project.referencedLibraries": [  
5         "lib/mysql-connector-j-9.7.0.jar",  
6         "lib/poi-5.2.5.jar",  
7         "lib/poi-ooxml-5.2.5.jar",  
8         "lib/poi-ooxml-lite-5.2.5.jar",  
9         "lib/commons-codec-1.16.0.jar",  
10        "lib/commons-collections4-4.4.jar",  
11        "lib/commons-io-2.15.0.jar",  
12        "lib/commons-math3-3.6.1.jar",  
13        "lib/commons-compress-1.26.1.jar",  
14        "lib/log4j-api-2.21.1.jar",  
15        "lib/log4j-core-2.21.1.jar",  
16        "lib/SparseBitSet-1.3.jar",  
17        "lib/xmlbeans-5.2.0.jar",  
18        "C:\\javafx-sdk-26.0.1\\lib\\*"  
19    ],  
20    "java.debug.settings.vmArgs": "--module-path \"C:\\javafx-sdk-26.0.1\\lib\" --add-modules javafx.controls,javafx.fxml"  
21 }
```

Listing 16 – settings.json - Configuration du projet

11.2 Dépendances principales

- MySQL Connector/J 9.7.0
- Apache POI 5.2.5 (Import/Export Excel)
- JavaFX 26.0.1
- Commons Codec 1.16.0
- Commons Collections 4.4
- Commons IO 2.15.0
- Commons Math3 3.6.1
- Commons Compress 1.26.1
- Log4j 2.21.1
- SparseBitSet 1.3
- XmlBeans 5.2.0

11.3 Structure des fichiers

```
.  
|-- modele/  
|   |-- Utilisateur.java  
|   |-- Etudiant.java
```



```
| |-- Enseignant.java
| |-- Promotion.java
| |-- Module.java
| |-- SousModule.java
| |-- Note.java
| |-- ReglementEvaluation.java
| |-- StockageMySQL.java
| |-- style.css
|-- InterfaceFx.java
|-- Principal.java
|-- InitDB.java
|-- lib/
| |-- mysql-connector-j-9.7.0.jar
| |-- poi-5.2.5.jar
| |-- poi-ooxml-5.2.5.jar
| |-- ...
|-- settings.json
```

11.4 Commandes utiles

11.4.1 Compilation avec JavaFX

```
javac --module-path "C:\javafx-sdk-26.0.1\lib" --add-modules javafx.controls,javafx.fxml
```

11.4.2 Exécution

```
java --module-path "C:\javafx-sdk-26.0.1\lib" --add-modules javafx.controls,javafx.fxml
```

11.4.3 Base de données MySQL

```
-- Création de la base de données
```

```
CREATE DATABASE gestion_notes;
```

```
-- Utilisation de la base de données
```

```
USE gestion_notes;
```

```
-- Voir toutes les tables
```

```
SHOW TABLES;
```

```
-- Voir les utilisateurs
```

```
SELECT * FROM utilisateur;
```